Terraform Complete Study & Revision Notes

Beginner to Production — AWS DevOps/SRE Edition

Aryan

August 2026

Contents

PART 1 — Infrastructure as Code (IaC)

What is IaC?

Infrastructure as Code means defining servers, networks, and cloud resources in **text files** instead of clicking through a console. The text is version-controlled, reviewed, and executed by a tool that talks to cloud APIs.

Why IaC is needed

- Manual (ClickOps) infra is **not repeatable** — two engineers building "the same" VPC by hand will produce slightly different results.
- No audit trail of *why* a resource exists.
- Scaling to 100s of resources by hand is error-prone and slow.
- IaC gives repeatability, peer review (PRs), versioning, and automation (CI/CD).

Declarative vs Imperative

	Declarative (Terraform)	Imperative (shell script / Ansible tasks)
You specify	**What** end state you want	**How** to get there, step by step
Execution	Tool computes the diff and steps	You write every step
Idempotency	Built-in	Must be hand-coded

Mutable vs Immutable Infrastructure

- **Mutable**: you SSH in and patch a running server. Drift accumulates over time ("snowflake servers").
- **Immutable**: you never patch a running instance — you build a new AMI/container and replace the instance. Terraform + Packer/Docker follow this model.

Desired State vs Current State

Terraform's whole job is reconciling three things:

```
Configuration (.tf files)   →  Desired State
Terraform State (.tfstate)  →  Last known state
Real Cloud Infrastructure   →  Actual State
```

`terraform plan` diffs Desired vs State vs Actual and shows you what will change.

Configuration Management vs Provisioning

- **Provisioning** (Terraform, CloudFormation, Pulumi): *creates* infrastructure — VPCs, EC2, RDS.
- **Configuration Management** (Ansible, Chef, Puppet): configures *what runs inside* a machine — installing packages, editing config files.
- They complement each other: Terraform creates the EC2 instance; Ansible configures the OS/app on it (or a Packer-built AMI replaces the need for it).

Terraform vs Alternatives

Tool	Type	Notes
Ansible	Imperative, agentless CM	Great for config management, weaker state tracking for infra
AWS CloudFormation	Declarative, AWS-only	No multi-cloud, native AWS integration, free
Pulumi	Declarative, real programming languages (TS/Python/Go)	Same HCL-vs-code tradeoff, real loops/conditionals natively
AWS CDK	Imperative code that *synthesizes* CloudFormation	AWS-only, generates CFN under the hood
Terraform	Declarative, multi-cloud, HCL	Huge provider ecosystem, mature state model

BEST PRACTICE: Use Terraform for provisioning infra; use Packer/Docker for images; use Ansible only if you truly need post-boot configuration (rare in immutable, container-first shops).

When NOT to use Terraform

- One-off, throwaway resources you'll delete in five minutes (just use the console/CLI).
- Extremely dynamic, app-level runtime scaling logic (that's the ASG/K8s HPA's job, not Terraform's).
- Anything requiring true imperative branching logic across many external systems in one pass (a general-purpose language / Pulumi may fit better).

The Terraform Workflow

```
You write HCL (.tf files)
        │
        ▼
  terraform init   (download providers/modules)
        │
        ▼
  terraform plan    (Config + State + Real infra → diff)
        │
        ▼
  terraform apply   (Provider → Cloud API → creates/updates infra)
        │
        ▼
  terraform state is updated to match new reality
```

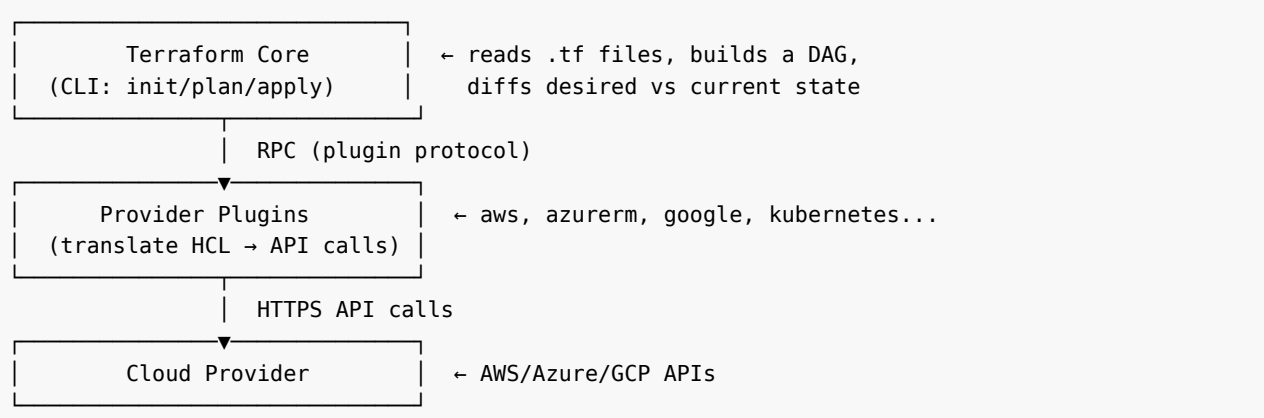
INTERVIEW TIP: "Explain the Terraform workflow" is one of the most common opening questions — always mention init → plan → apply → state update, and that Terraform is declarative.

PART 2 — Terraform Fundamentals

What is Terraform?

Terraform (by HashiCorp) is an open-source, declarative IaC tool that uses **HCL** (HashiCorp Configuration Language) to define infrastructure across 3000+ providers (AWS, Azure, GCP, Kubernetes, GitHub, Datadog, etc.) and reconciles it via a single workflow.

Terraform Architecture



Key File Types

File	Purpose
`*.tf`	Your HCL configuration (providers, resources, variables...)
`*.tfvars`	Values for input variables
`terraform.tfstate`	JSON file mapping your config to real resource IDs
`.terraform/`	Downloaded provider plugins + module cache (local, gitignored)
`.terraform.lock.hcl`	Locked provider *versions* + checksums — **commit this**

Project Structure (beginner)

```
terraform/
├── main.tf
├── variables.tf
├── outputs.tf
└── terraform.tfvars
```

Core CLI Commands (what happens internally)

Command	What it does internally
`terraform init`	Reads `required_providers`, downloads plugins into `.terraform/`, initializes backend, creates/updates the lock file
`terraform validate`	Parses HCL syntax + internal consistency (types, required args) — **no API calls**
`terraform fmt`	Rewrites files to canonical style (indentation, alignment)
`terraform plan`	Refreshes state (reads real infra via provider), builds dependency graph, diffs config vs state vs real infra, prints an execution plan
`terraform apply`	Re-runs plan (or uses a saved plan file), asks for approval, walks the DAG calling Create/Update/Delete APIs, writes new state
`terraform destroy`	Same as apply but the target state is "nothing exists"
`terraform show`	Pretty-prints the state file or a saved plan file
`terraform output`	Reads output values from state
`terraform refresh`	(legacy — folded into `plan`/`apply -refresh-only`) syncs state with real infra without changing config
`terraform providers`	Shows the provider dependency tree
`terraform version`	Shows Terraform + provider versions

COMMON MISTAKE: Running `terraform apply` in production without ever running `plan` first and reading it. Always read the plan — it's your safety net.

PART 3 — HCL Complete Guide

Syntax basics

HCL is built from **blocks** and **arguments**:

```
<block_type> "<label1>" "<label2>" {
  argument_name = value
}
```

Example:

```
resource "aws_instance" "web" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t3.micro"
}
```

- `resource` = block type
- `"aws_instance"` = the resource *type* label
- `"web"` = the local *name* label (your name for it)
- `ami`, `instance_type` = arguments

Data Types

Type	Example
`string`	`"hello"`
`number`	`42`, `3.14`
`bool`	`true`, `false`
`list(type)`	`["a", "b", "c"]` — ordered, allows duplicates
`set(type)`	unordered, unique values
`map(type)`	`{ key = "value" }`
`object({...})`	structured, named attributes with fixed types
`tuple([...])`	fixed-length, each element can be a different type
`null`	absence of a value

```
variable "az_list" {
  type    = list(string)
  default = ["ap-south-1a", "ap-south-1b"]
}

variable "tags" {
  type = map(string)
  default = {
    Environment = "dev"
    Owner       = "aryan"
  }
}

variable "server" {
  type = object({
    name = string
    cpu  = number
    tags = list(string)
```

```
  })
}
```

Comments

```
# single line
// also single line
/* multi
   line */
```

Strings & Interpolation

```
name = "web-${var.environment}-${count.index}"
```

Heredoc (multi-line strings — common for user_data, IAM policy JSON):

```
user_data = <<-EOF
              #!/bin/bash
              echo "Hello from ${var.environment}" > /var/www/html/index.html
            EOF
```

Operators

- Arithmetic: `+ - * / %`
- Comparison: `== != < > <= >=`
- Logical: `&& || !`

Conditional Expression

```
instance_type = var.environment == "prod" ? "t3.large" : "t3.micro"
```

`for` Expressions

```
# list → list
[for s in var.subnet_ids : upper(s)]

# list → map
{for s in var.servers : s.name => s.ip}

# with filtering
[for s in var.subnets : s.id if s.public == true]
```

Splat Expressions

```
aws_instance.web[*].id      # all instance IDs from a count/for_each resource
```

References

```
var.instance_type          # a variable
local.name_prefix          # a local value
aws_instance.web.id        # a resource attribute
data.aws_ami.latest.id     # a data source attribute
module.vpc.vpc_id          # a module output
```

INTERVIEW TIP: Know the difference between `list`/`tuple` (ordered) and `set` (unordered, unique) — a very common “gotcha” question, especially around `for_each` requiring a **set or map**, not a list.

PART 4 — Terraform Blocks

`terraform` block

Configures Terraform itself: required version, required providers, backend.

```
terraform {
  required_version = ">= 1.7.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket = "my-tfstate-bucket"
    key    = "prod/network/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

`provider` block

Configures a specific provider (credentials, region, default tags).

```
provider "aws" {
  region = "ap-south-1"
  default_tags {
    tags = { ManagedBy = "Terraform" }
  }
}
```

`resource` block

Declares a piece of infrastructure to create/manage.

```
resource "aws_s3_bucket" "logs" {
  bucket = "my-app-logs-bucket"
}
```

`data` block

Reads existing (not-managed-by-this-config) infrastructure.

```
data "aws_vpc" "default" {
  default = true
}
```

`variable` block

Declares an input.

```
variable "region" {
  type    = string
  default = "ap-south-1"
}
```

`locals` block

Declares computed/reusable values.

```
locals {
  name_prefix = "${var.project}-${var.environment}"
}
```

`output` block

Exposes a value after apply.

```
output "vpc_id" {
  value = aws_vpc.main.id
}
```

`module` block

Calls a reusable module.

```
module "vpc" {
  source = "./modules/vpc"
  cidr   = "10.0.0.0/16"
}
```

`moved` block (refactor safely, Terraform ≥1.1)

```
moved {
  from = aws_instance.old_name
  to   = aws_instance.new_name
}
```

`import` block (Terraform ≥1.5 — declarative import)

```
import {
  to = aws_s3_bucket.logs
  id = "my-existing-bucket-name"
}
```

`check` block (Terraform ≥1.5 — post-apply health assertions)

```
check "website_up" {
  data "http" "homepage" {
    url = "https://example.com"
  }
  assert {
```

```
    condition     = data.http.homepage.status_code == 200
    error_message = "Website did not return 200"
  }
}
```

`lifecycle` block (nested inside `resource`)

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true
    prevent_destroy        = false
    ignore_changes          = [tags]
  }
}
```

Note on "ephemeral": Terraform 1.10+ introduced *ephemeral resources/values* — data that exists only during a plan/apply (e.g., a short-lived DB password) and is never written to state. Useful for secrets pulled from Vault/Secrets Manager that shouldn't persist in `.tfstate`.

PART 5 — Providers

What is a provider?

A plugin (a separate compiled binary) that translates HCL resource blocks into API calls for a specific platform (AWS, Azure, Kubernetes, GitHub…). Terraform Core knows nothing about AWS — the `aws` provider does.

Provider configuration & version constraints

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"   # registry.terraform.io/hashicorp/aws
      version = "~> 5.40"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}
```

Constraint	Meaning
`= 5.40.0`	Exact version
`>= 5.0`	Minimum
`<= 5.50`	Maximum
`~> 5.40`	`>= 5.40.0, < 5.41.0` (patch-level flexibility)
`~> 5.0`	`>= 5.0.0, < 6.0.0` (minor-level flexibility)

WARNING: Never leave a provider unpinned in production — an unannounced major version bump can silently change resource behavior or defaults.

Provider Aliases (multi-region / multi-account)

```
provider "aws" {
  region = "ap-south-1"
}

provider "aws" {
  alias  = "us_east"
  region = "us-east-1"
}

resource "aws_acm_certificate" "cf_cert" {
  provider = aws.us_east   # CloudFront certs must be in us-east-1
  # ...
}
```

The Lock File

`.terraform.lock.hcl` records exact provider versions + cryptographic hashes so every teammate/CI run gets **identical** provider binaries. Generated/updated by `terraform init` (or `terraform providers lock`). **Always commit it.**

COMMON MISTAKE: Deleting `.terraform.lock.hcl` "to fix an error" — this can silently upgrade providers and change behavior. Instead run `terraform init -upgrade` deliberately when you *want* to bump versions.

PART 6 — Variables

Declaration

```
variable "instance_type" {
  type        = string
  description = "EC2 instance type"
  default     = "t3.micro"
  sensitive   = false
}
```

Validation blocks

```
variable "environment" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}
```

Complex types

```
variable "servers" {
  type = list(object({
    name = string
    cpu  = number
  }))
}

variable "tags_by_env" {
  type = map(object({
    instance_type = string
    min_size      = number
  }))
}
```

Sensitive variables

```
variable "db_password" {
  type      = string
  sensitive = true   # hides value from plan/apply CLI output (still in state!)
}
```

WARNING: `sensitive = true` only hides the value from CLI output — it is **not encrypted** inside `terraform.tfstate`. Protect state itself (encrypted backend + strict IAM) for real secrets, or better, use ephemeral resources / a secrets manager reference.

Setting variable values — precedence (highest wins)

1. `-var` / `-var-file` CLI flags (last one on the command line wins among these)

2. `*.auto.tfvars` / `*.auto.tfvars.json` (alphabetical order)
3. `terraform.tfvars` / `terraform.tfvars.json`
4. `TF_VAR_<name>` environment variables
5. `default` in the variable block (lowest)

```
export TF_VAR_db_password="s3cr3t"
terraform apply -var="instance_type=t3.large" -var-file="prod.tfvars"
```

BEST PRACTICE: Commit `*.tfvars.example`, never commit real `*.tfvars` containing secrets; use `TF_VAR_*` or a secrets manager + CI/CD injected variables instead.

PART 7 — Locals

Locals are named expressions computed once per plan/apply — think of them as “variables you can’t change from outside,” used for DRY-ing up repeated expressions.

```
locals {
  name_prefix = "${var.project}-${var.environment}"
  common_tags = {
    Project     = var.project
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
}

resource "aws_instance" "web" {
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-web" })
}
```

	Variables	Locals	Outputs
Set by	Caller (tfvars/CLI/env)	Computed inside the module	Exposed to caller
Direction	Input	Internal	Output
Can reference resources?	No	Yes	Yes

PART 8 — Outputs

```
output "vpc_id" {
  description = "ID of the created VPC"
  value       = aws_vpc.main.id
}

output "db_password" {
  value     = aws_db_instance.main.password
  sensitive = true
}
```

- Root module outputs are shown after `terraform apply` and via `terraform output`.
- Child module outputs must be explicitly re-exposed by the parent if the root needs them: `module.vpc.vpc_id`.
- Outputs can create *implicit dependencies* — referencing a resource in an output means Terraform won't consider apply "done" for dependents until that resource exists.

PART 9 — Resources

```
resource "aws_instance" "web" {
  ami           = var.ami                # argument
  instance_type = var.instance_type      # argument

  tags = {
    Name = "web-server"
  }
}
```

- **Resource address**: `aws_instance.web` (type.name) — used everywhere: state commands, `-target`, references.
- **Attributes**: after creation, computed values like `aws_instance.web.id`, `.private_ip`, `.arn` become available to other resources.

Dependencies

- **Implicit**: Terraform auto-detects order when one resource references another's attribute.

```
resource "aws_subnet" "public" {
  vpc_id = aws_vpc.main.id   # implicit dependency on aws_vpc.main
}
```

- **Explicit**: use `depends_on` when there's a dependency Terraform *can't see* (e.g., IAM eventual consistency, side effects not expressed via attributes).

```
resource "aws_instance" "app" {
  # ...
  depends_on = [aws_iam_role_policy.app_policy]
}
```

Resource lifecycle (create/update/replace/destroy)

Terraform decides per-attribute whether a change can be an **in-place update** or requires **replacement** (destroy + create) — this is defined by the *provider* (some attributes are marked `ForceNew`). `terraform plan` shows this with `-/+`.

COMMON MISTAKE: Changing an attribute that forces replacement (e.g., `availability_zone` on some resources) without realizing the resource will be destroyed and recreated — always read plan output symbols carefully.

PART 10 — Data Sources

Data sources **read** existing infrastructure (created by you manually, by another team, or by another Terraform state) without managing its lifecycle.

```
data "aws_ami" "latest_amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

data "aws_vpc" "default" {
  default = true
}

data "aws_availability_zones" "available" {
  state = "available"
}

data "aws_caller_identity" "current" {}   # who/what account am I running as?

resource "aws_instance" "web" {
  ami               = data.aws_ami.latest_amazon_linux.id
  availability_zone = data.aws_availability_zones.available.names[0]
}
```

	`resource`	`data`
Lifecycle	Terraform creates/updates/destroys it	Terraform only reads it
In state?	Yes, fully	Yes, but as a read cache
Use case	New infra you own	Existing infra / computed AWS values (AMIs, AZs, account ID)

PART 11 — Meta-Arguments

`count`

```
resource "aws_instance" "web" {
  count         = 3
  ami           = var.ami
  instance_type = var.instance_type
  tags          = { Name = "web-${count.index}" }
}
# reference: aws_instance.web[0].id, aws_instance.web[*].id
```

`for_each`

```
resource "aws_instance" "web" {
  for_each      = toset(["a", "b", "c"])
  ami           = var.ami
  instance_type = var.instance_type
  tags          = { Name = "web-${each.key}" }
}
# reference: aws_instance.web["a"].id
```

`count` vs `for_each` — the critical difference

	`count`	`for_each`
Index type	Numeric (0,1,2..)	String key (map/set)
Reordering safety	**Unsafe** — removing item 1 of 3 shifts indices 2,1 and can force-replace unrelated resources	**Safe** — each resource is tied to a stable key, unaffected by other keys changing
Best for	Truly identical, interchangeable resources (e.g., N identical NAT gateways)	Resources that differ per item or where identity must be stable (e.g., IAM users per name)

BEST PRACTICE: Default to `for_each` with a map/set whenever items have a natural unique key (name, ID) — avoids the classic “changed instance 2’s tags, Terraform wants to destroy instance 3” bug caused by `count` re-indexing.

`depends_on` (also usable on modules)

```
module "app" {
  source     = "./modules/app"
  depends_on = [module.vpc]
}
```

`provider` meta-argument

```
resource "aws_acm_certificate" "cert" {
  provider = aws.us_east
}
```

lifecycle

Covered in Part 21.

PART 12 — Expressions and Functions

Commonly used built-in functions

Function	Purpose	Example
`merge(a, b)`	Combine maps (later wins on conflict)	`merge(local.tags, {Name="x"})`
`concat(a, b)`	Combine lists	`concat(["a"], ["b"])` → `["a","b"]`
`flatten(list)`	Flatten nested lists	`flatten([["a"],["b","c"]])`
`join(sep, list)`	List → string	`join(",", ["a","b"])` → `"a,b"`
`split(sep, str)`	String → list	`split(",", "a,b")`
`lookup(map, key, default)`	Safe map access	`lookup(var.sizes, "prod", "t3.micro")`
`contains(list, val)`	Membership test	`contains(var.envs, "prod")`
`length(x)`	Size of list/map/string	`length(var.subnets)`
`keys(map)` / `values(map)`	Extract keys/values	
`toset()/tolist()/tomap()`	Type conversion	
`try(expr, default)`	Attempt, fall back on error	`try(var.x.y, "fallback")`
`can(expr)`	Boolean: did expr succeed?	used inside `validation` blocks
`coalesce(a, b, c)`	First non-null value	
`distinct(list)`	Remove duplicates	
`sort(list)`	Alphabetical sort	
`element(list, i)`	Index with wraparound	
`cidrsubnet(prefix, newbits, netnum)`	Compute a subnet CIDR	`cidrsubnet("10.0.0.0/16", 8, 1)` → `10.0.1.0/24`
`jsonencode()/jsondecode()`	JSON ↔ HCL	used heavily for IAM policies
`timestamp()` / `formatdate()`	Date/time	
`file()` / `templatefile()`	Read local files, render templates	user_data scripts
`sha256()/md5()/uuid()`	Hash/crypto	

Real use case — building IAM policy JSON

```
data "aws_iam_policy_document" "s3_read" {
  statement {
    actions   = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.data.arn}/*"]
  }
}

resource "aws_iam_policy" "s3_read" {
  policy = data.aws_iam_policy_document.s3_read.json
}
```

PART 13 — Dynamic Blocks

Used to generate *repeated nested blocks* (like multiple `ingress` rules inside a security group) from a list/map, instead of hardcoding each one.

```
variable "ingress_rules" {
  type = list(object({
    port        = number
    cidr_blocks = list(string)
  }))
  default = [
    { port = 22, cidr_blocks = ["10.0.0.0/16"] },
    { port = 80, cidr_blocks = ["0.0.0.0/0"] },
  ]
}

resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = "tcp"
      cidr_blocks = ingress.value.cidr_blocks
    }
  }
}
```

COMMON MISTAKE: Overusing dynamic blocks for things that rarely change — they hurt readability. Use them only when the number/shape of nested blocks genuinely varies by input.

PART 14 — Dependencies & the DAG

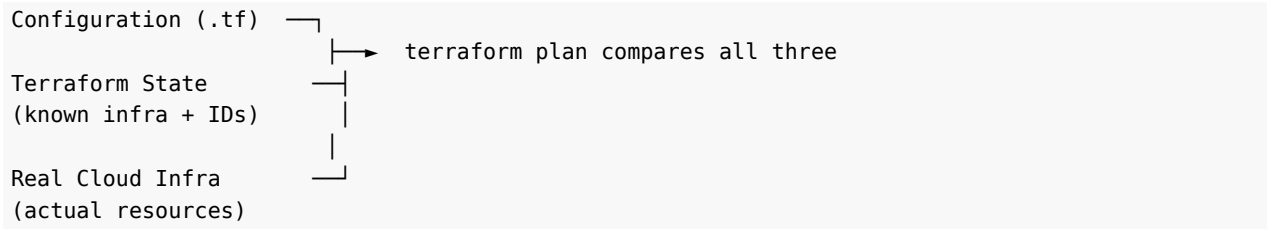
Terraform builds a **Directed Acyclic Graph (DAG)** from every implicit (attribute reference) and explicit (`depends_on`) dependency, then: 1. Walks the graph to determine safe ordering. 2. Executes **independent** resources **in parallel** (default parallelism = 10, tune with `-parallelism=n`). 3. Refuses to apply if the graph has a cycle (e.g., A depends on B which depends on A) — this is a config error you must fix.

```
terraform graph | dot -Tpng > graph.png   # requires Graphviz
```

PART 15 — Terraform State (Deep Dive)

Why state exists

Terraform is declarative — your `.tf` files describe *what you want*, not resource IDs. Terraform needs a way to map `aws_instance.web` → the real AWS instance `i-0abc123...`. That mapping, plus cached attribute values, **is** the state file.



Without state, Terraform would have to re-discover every resource from scratch on every run (slow, ambiguous, and impossible for resources with no unique queryable attributes).

What's inside `terraform.tfstate`

A JSON file containing: - Terraform + provider version metadata - A `serial` number (increments every write) and a `lineage` UUID (identifies this state's "family" — used to detect swapped/wrong state files) - Every managed resource: its address, attributes, dependencies, and provider

WARNING: State files can contain **plaintext secrets** (e.g., an RDS password set via a resource argument) even if the variable was marked `sensitive`. Never commit state to Git; treat the backend like a secrets store.

State Drift

Drift = the real infrastructure changed **outside of Terraform** (someone edited a security group in the console, auto-scaling changed instance count, etc.), so State ≠ Real Infra. - `terraform plan` automatically **refreshes** and will show drift as a diff. - `terraform apply -refresh-only` updates state to match reality **without** changing infra — use this to consciously "accept" drift into state. - Prevention: IAM permissions that block console changes to Terraform-managed resources; `prevent_destroy`; tagging conventions; policy-as-code (OPA/Sentinel) to block manual changes.

State Locking

When someone runs `plan`/`apply`, Terraform acquires a **lock** on the state so a second concurrent run can't corrupt it. With an S3 backend, this is done via a companion DynamoDB table (or, in newer Terraform/S3 setups, S3's native conditional-write locking — see Part 17). Local state uses an OS-level file lock.

State Security Checklist

- Store remotely (S3, Terraform Cloud, Azure Blob, GCS) — never rely on a laptop's local `terraform.tfstate`.
- Enable **encryption at rest** (SSE-KMS on S3).
- Enable **versioning** on the state bucket (instant rollback if corrupted).
- Restrict IAM to only the CI/CD role + admins.
- Never commit `.tfstate` to Git — add it to `.gitignore`.

PART 16 — State Commands

Command	Purpose	When to use / avoid
`terraform state list`	List all resource addresses in state	Safe, read-only — use anytime to explore
`terraform state show <addr>`	Show full attributes of one resource	Debugging, safe
`terraform state mv <src> <dst>`	Rename/move a resource address in state without destroying it	Refactoring (renaming a resource, moving into a module) — **prefer the `moved` block in modern Terraform for reviewable, repeatable refactors**
`terraform state rm <addr>`	Remove a resource from state **without** destroying real infra	When you want Terraform to "forget" a resource (e.g., handing it to another team) — real infra is untouched
`terraform state pull`	Print raw state JSON (from remote backend)	Scripting/inspection
`terraform state push`	Force-overwrite remote state	**Dangerous** — only for manual disaster recovery, never routine use

WARNING: `terraform state rm` does not delete the actual AWS resource — only the mapping. If you then re-`apply` without importing it back, Terraform will try to create a *duplicate* resource.

PART 17 — Remote Backend

What is a backend?

The backend determines **where** state is stored and **how** it's locked. Local backend (default) stores it on disk — fine for solo learning, unsafe for teams.

S3 Backend (AWS production standard)

```
terraform {
  backend "s3" {
    bucket         = "my-company-tfstate"
    key            = "prod/networking/terraform.tfstate"
    region         = "ap-south-1"
    encrypt        = true
    dynamodb_table = "terraform-locks"   # classic locking mechanism
  }
}
```

```
Terraform  ──►  S3 bucket (state storage, versioned, encrypted)
           ──►  DynamoDB table (state locking, prevents concurrent apply)
```

Modern note (Terraform ≥1.10 / AWS provider ≥5.x with S3 native locking): AWS added native **conditional writes** to S3, and Terraform/the S3 backend added support for locking directly via S3 (`use_lockfile = true`) without a separate DynamoDB table. DynamoDB locking still works and is extremely common in existing production setups — know both, but prefer the S3-native approach for new projects to simplify your architecture.

Backend init & migration

```
terraform init                    # first-time backend init
terraform init -migrate-state     # switching from local → remote (or backend config change)
terraform init -reconfigure       # reinitialize without migrating old state
```

BEST PRACTICE: One state file per logical "blast radius" (e.g., separate state for networking vs compute vs database) rather than one giant state for the whole org — smaller state = faster plans, safer applies, easier team ownership.

PART 18 — State Drift (Scenarios)

Scenario: An engineer manually widened a security group rule in the AWS console to unblock an urgent incident. 1. Next `terraform plan` shows a diff reverting the console change back to the `.tf` definition (Terraform doesn't know about "why", only "what"). 2. **Fix path A** — encode the new rule into `.tf` and commit it (make the code match reality). 3. **Fix path B** — if the console change was a mistake, just `apply` to revert it. 4. To *accept* current reality into state without changing infra: `terraform apply -refresh-only`.

BEST PRACTICE: Restrict console write-access to Terraform-managed resources via IAM/SCP so drift is rare and intentional (e.g., only allow break-glass roles with mandatory follow-up code updates).

PART 19 — Modules

What is a module?

Any directory containing `.tf` files is a module. The directory you run `terraform apply` in is the **root module**; anything it calls via a `module` block is a **child module**.

Structure

```
modules/
├── vpc/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
├── security-group/
├── ec2/
├── alb/
└── rds/
```

modules/vpc/main.tf

```
resource "aws_vpc" "this" {
  cidr_block = var.cidr
  tags       = { Name = var.name }
}

resource "aws_subnet" "public" {
  for_each          = var.public_subnets
  vpc_id            = aws_vpc.this.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az
}
```

modules/vpc/variables.tf

```
variable "cidr" { type = string }
variable "name" { type = string }
variable "public_subnets" {
  type = map(object({ cidr = string, az = string }))
}
```

modules/vpc/outputs.tf

```
output "vpc_id"            { value = aws_vpc.this.id }
output "public_subnet_ids" { value = [for s in aws_subnet.public : s.id] }
```

Root module calling it

```
module "vpc" {
  source = "./modules/vpc"
  cidr   = "10.0.0.0/16"
```

```
  name   = "prod-vpc"
  public_subnets = {
    a = { cidr = "10.0.1.0/24", az = "ap-south-1a" }
    b = { cidr = "10.0.2.0/24", az = "ap-south-1b" }
  }
}

module "ec2" {
  source = "./modules/ec2"
  subnet_id = module.vpc.public_subnet_ids[0]   # inter-module dependency
}
```

Module sources

```
source = "./modules/vpc"                                          # local
source = "git::https://github.com/org/tf-modules.git//vpc?ref=v1.2.0"  # git w/ version pin
source = "terraform-aws-modules/vpc/aws"                          # Terraform Registry
version = "~> 5.0"                                                 # registry version constraint
```

BEST PRACTICE: Pin module versions the same way you pin providers — `?ref=v1.2.0` for Git, `version = "~>"` for registry modules. Never point production at a branch like `?ref=main`.

PART 20 — Module Best Practices

- **Single responsibility**: a `vpc` module shouldn't also create EC2 instances.
- **Sane, documented interface**: required inputs minimal, optional inputs have sensible defaults; document every variable/output with `description`.
- **Don't over-parameterize**: a module with 80 input variables to handle every edge case becomes unmaintainable — prefer composition (small modules combined) over one mega-module.
- **Version everything**: modules, providers, and the module's own `required_providers`.
- **Test modules** in isolation (see Part 42 — `terraform test`).
- **Public vs private**: use the public Terraform Registry for generic infra (VPC, EKS) from trusted maintainers (e.g., `terraform-aws-modules/*`); use a **private registry** (Terraform Cloud/Artifactory/Git) for company-specific modules with internal conventions baked in.

INTERVIEW TIP: "Why use modules?" → reusability, consistency across environments, encapsulation (hide complexity behind a clean interface), and easier testing/versioning.

PART 21 — Lifecycle Management

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true
    prevent_destroy        = true
    ignore_changes          = [tags, user_data]
    replace_triggered_by    = [aws_launch_template.web.id]
  }
}
```

Setting	Purpose	Risk
`create_before_destroy`	Build the replacement **before** destroying the old one (zero-downtime swaps, e.g., launch templates behind an ASG)	Can briefly duplicate resources with unique-name constraints unless names are randomized
`prevent_destroy`	Hard-block `terraform destroy`/replace on this resource (e.g., production RDS)	Must be manually removed from code before you *can* delete it — that's the point
`ignore_changes`	Tell Terraform to stop diffing specific attributes (e.g., autoscaler-managed `desired_capacity`)	Overuse hides real drift you should know about
`replace_triggered_by`	Force replacement when a *referenced* resource/attribute changes, even if this resource's own args didn't	Used for rolling instance refresh when an AMI changes
`precondition` / `postcondition` (nested `lifecycle` blocks, ≥1.2)	Assert an invariant before/after apply	Fails the apply loudly instead of silently deploying bad config

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    postcondition {
      condition     = self.public_ip != ""
      error_message = "Instance did not get a public IP."
    }
  }
}
```

PART 22 — Importing Existing Infrastructure

Why import?

Real teams almost never start from zero — some infra already exists (built by hand, by another tool, or before Terraform adoption). Import lets Terraform "adopt" it into state without recreating it.

Modern workflow — `import` block (Terraform ≥1.5, preferred)

```
import {
  to = aws_s3_bucket.logs
  id = "my-existing-bucket-name"
}

resource "aws_s3_bucket" "logs" {
  bucket = "my-existing-bucket-name"
}
```

Run `terraform plan` — it shows what config would be generated / any diff, then `terraform apply` performs the import. You can also generate a starting config automatically:

```
terraform plan -generate-config-out=generated.tf
```

Legacy workflow — CLI import

```
terraform import aws_s3_bucket.logs my-existing-bucket-name
```

You must **already have** (or immediately write) a matching `resource` block — CLI import only populates state, it does not write HCL for you (unlike `-generate-config-out`).

COMMON MISTAKE: Importing into state but writing a resource block that doesn't match real attributes — next `plan` will show a big (sometimes destructive) diff. Always run `plan` immediately after import and reconcile any differences into your `.tf` before touching `apply`.

PART 23 — Resource Moving & Refactoring

`moved` block (preferred, reviewable in PRs)

```
moved {
  from = aws_instance.web
  to   = aws_instance.app
}
```

Renaming or moving into a module:

```
moved {
  from = aws_instance.web
  to   = module.ec2.aws_instance.web
}
```

`terraform state mv` (imperative equivalent)

```
terraform state mv aws_instance.web aws_instance.app
terraform state mv aws_instance.web module.ec2.aws_instance.web
```

BEST PRACTICE: Prefer `moved` blocks over ad-hoc `state mv` commands — they're committed to Git, applied automatically by anyone running `apply` (including CI), and self-document *why* an address changed. `state mv` is a one-off, manual, easy-to-forget-on-a-teammate's-machine operation.

PART 24 — Terraform Plan

```
terraform plan
terraform plan -out=tfplan       # save an exact plan to apply later
terraform show tfplan            # inspect a saved plan
terraform apply tfplan           # apply EXACTLY what was planned (no surprises from drift between plan and ap
```

Plan symbols

Symbol	Meaning
`+`	Create
`-`	Destroy
`~`	Update in place
`-/+`	Destroy and re-create (replacement)
`<=`	Read (data source)

BEST PRACTICE (CI/CD): Always `plan -out=tfplan` in CI, post it for human review, then `apply tfplan` — this guarantees what gets approved is *exactly* what gets applied, with no time-of-check/time-of-use gap.

PART 25 — Terraform Apply

- Default: prompts `yes` confirmation after showing the plan.
- `-auto-approve`: skips confirmation — **only** safe in CI pipelines with a prior human-reviewed plan gate, never for ad-hoc local production applies.
- `-target=<addr>`: applies only a subset of the graph.

WARNING — why `-target` should not be routine: it applies a *partial* graph, which can leave state inconsistent with config (resources that depend on the targeted one aren't updated), and hides real dependency problems instead of fixing them. Use it only for genuine emergency/recovery situations, then immediately run a full `plan` to reconcile.

Failure handling

If `apply` fails partway through (e.g., API error on resource #7 of 10), Terraform: - Has already updated state for resources #1–6 (they exist and are tracked). - Stops and reports the error for #7. - On the next `apply`, it resumes from where it left off (only the failed/remaining resources are attempted).

PART 26 — Terraform Destroy

```
terraform destroy
terraform destroy -target=aws_instance.web   # narrow scope (same -target caveats apply)
```

- Respects `prevent_destroy` and dependency ordering (destroys leaf/dependent resources before the resources they depend on).
- **Production risk:** an accidental `destroy` in the wrong workspace/directory is one of the most common catastrophic DevOps incidents.

Safer alternatives: - `prevent_destroy = true` on critical resources (RDS, S3 with data). - Separate state per environment so a `dev` destroy can't touch `prod`. - Require manual approval + MFA/break-glass role for any destroy in CI. - `terraform plan -destroy` to preview first.

PART 27 — Terraform Workspaces

```
terraform workspace list
terraform workspace new staging
terraform workspace select staging
terraform workspace show
```

Workspaces let one config directory manage multiple **state files** (`terraform.tfstate.d/<name>/terraform.tfstate`) using `terraform.workspace` in code:

```
resource "aws_instance" "web" {
  instance_type = terraform.workspace == "prod" ? "t3.large" : "t3.micro"
}
```

Workspaces vs alternatives

Approach	Isolation strength	Best for
Workspaces	Same backend/credentials, different state file	Small variations (e.g., feature branches, ephemeral test envs)
Directory-based (`environments/dev`, `environments/prod`)	Fully separate config + state, explicit per-env `.tfvars`	Most production setups — clear, explicit, reviewable diffs between envs
Separate AWS accounts	Full blast-radius isolation (IAM, billing, limits)	**Best practice for prod vs non-prod** at any real company

WARNING: Workspaces are *not* a substitute for account isolation — a `prod` workspace still shares the same AWS account/credentials as `dev` unless you also vary the provider config, which reintroduces complexity. Most production shops prefer directory-based environments + separate AWS accounts over workspaces alone.

PART 28 — Environment Management

Recommended production pattern:

```
environments/
├── dev/       (own state, own tfvars, targets Dev AWS account)
├── staging/   (own state, own tfvars, targets Staging AWS account)
└── prod/      (own state, own tfvars, targets Prod AWS account)

modules/       (shared, versioned building blocks used by all three)
```

Each environment directory has its own `backend` config (different S3 key, possibly different bucket/account) and its own `terraform.tfvars`, but calls the **same shared modules** — so dev/staging/prod stay structurally consistent while allowing per-env sizing/scaling differences.

CI/CD enforces environment promotion: a change merges to `dev` first, is validated, then promoted (via PR or pipeline stage) to `staging`, then `prod` — never applied to `prod` first.

Terragrunt (a popular wrapper, not covered by core Terraform) is often introduced at this stage to DRY up repeated backend/provider boilerplate across many environment directories — worth knowing the *name* and *purpose* for interviews even if you haven't used it yet.

PART 29 — Terraform Registry

- **Provider Registry** (`registry.terraform.io/providers/...`): where `hashicorp/aws`, `hashicorp/azurerm`, etc. are published; referenced via `source` in `required_providers`.
- **Module Registry**: public, versioned, reusable modules (e.g., `terraform-aws-modules/vpc/aws`) — semantic-versioned, so `version = "~> 5.0"` behaves like a provider constraint.
- **Private Registry** (via Terraform Cloud/Enterprise, or self-hosted): internal modules with the same versioning UX, restricted to your org.

BEST PRACTICE: Prefer well-maintained community modules (e.g., the `terraform-aws-modules` GitHub org) for generic, "solved" infra like VPCs — don't reinvent a VPC module from scratch unless you have a strong reason.

PART 30 — HCP Terraform / Terraform Cloud

HashiCorp's managed SaaS (now branded **HCP Terraform**, formerly Terraform Cloud) adds a control plane on top of the open-source CLI:

Concept	What it does
Workspaces (HCP, different from CLI workspaces)	Each maps to one config + one state, with its own variables and run history
Remote execution	`plan`/`apply` run on HashiCorp's infrastructure, not your laptop/CI runner
Remote state	Built-in, versioned, locked — no need to hand-roll an S3 backend
Variable Sets	Share variables (e.g., AWS creds) across many workspaces
VCS integration	Auto-plan on PR open, auto-apply on merge
Sentinel / OPA policy	Policy-as-code gates ("no public S3 buckets") enforced before apply
Private Registry	Host internal modules/providers
Cost estimation	Shows $ impact of a plan before apply
Teams & permissions	RBAC over workspaces

Distinguish clearly: Terraform **CLI** (`terraform` binary) is free, open-source, and works with any backend (including HCP Terraform's remote backend). **HCP Terraform** is the optional managed platform that *orchestrates* CLI runs and adds governance — you can use 100% CLI + self-hosted S3 backend and never touch HCP Terraform at all, which is common at many companies.

PART 31 — Terraform CLI Complete Reference

Command	Purpose	Example
`terraform init`	Initialize working dir, download providers/modules, set up backend	`terraform init -upgrade`
`terraform validate`	Check config syntax/internal consistency	`terraform validate`
`terraform fmt`	Auto-format `.tf` files	`terraform fmt -recursive`
`terraform plan`	Show execution plan	`terraform plan -out=tfplan`
`terraform apply`	Apply changes	`terraform apply tfplan`
`terraform destroy`	Destroy managed infra	`terraform destroy -target=aws_instance.web`
`terraform show`	Human-readable state/plan	`terraform show tfplan`
`terraform output`	Print output values	`terraform output vpc_id`
`terraform refresh`	*(legacy, use `apply -refresh-only`)* Sync state with real infra	
`terraform providers`	Show provider requirements tree	`terraform providers`
`terraform version`	Show TF + provider versions	
`terraform graph`	Print dependency graph (DOT format)	`terraform graph | dot -Tpng > g.png`
`terraform console`	Interactive expression REPL	`terraform console`
`terraform get`	Download/update modules only	`terraform get -update`
`terraform force-unlock`	Manually clear a stuck state lock	`terraform force-unlock <LOCK_ID>`
`terraform state ...`	State inspection/surgery (Part 16)	
`terraform import`	Legacy imperative import	`terraform import aws_s3_bucket.b my-bucket`
`terraform workspace ...`	Manage workspaces (Part 27)	
`terraform taint` *(deprecated since 0.15.2)*	Force a resource to be replaced next apply	Use `terraform apply -replace=<addr>` instead
`terraform untaint` *(deprecated)*	Undo a taint	Use `-replace` workflow instead
`terraform test`	Run `.tftest.hcl` test files (Part 42)	`terraform test`

DEPRECATED — modern alternative: `taint`/`untaint` are deprecated; use:

```
terraform apply -replace="aws_instance.web"
```

PART 32 — Terraform Console

An interactive REPL for evaluating expressions against your *current state* — invaluable for debugging without a real apply.

```
terraform console
> var.instance_type
"t3.micro"
> aws_instance.web.private_ip
"10.0.1.15"
> cidrsubnet("10.0.0.0/16", 8, 2)
"10.0.2.0/24"
> [for s in aws_subnet.public : s.id]
```

BEST PRACTICE: Use `console` to test a tricky `for` expression or function chain before pasting it into a resource block — far faster than repeated `plan` cycles.

PART 33 — Terraform Graph

```
terraform graph > graph.dot
dot -Tpng graph.dot -o graph.png     # requires Graphviz's `dot`
```

Outputs the full dependency DAG in Graphviz DOT format — useful for visually understanding *why* Terraform is ordering things a certain way, or diagnosing an unexpected `depends_on` chain.

PART 34 — Debugging and Logging

```
export TF_LOG=DEBUG          # TRACE > DEBUG > INFO > WARN > ERROR
export TF_LOG_PATH=./tf.log  # write logs to a file instead of stdout
terraform apply
```

Level	Use
`TRACE`	Maximum verbosity — provider-level RPC calls, very noisy
`DEBUG`	Most useful day-to-day debugging level
`INFO`	High-level operational messages
`WARN` / `ERROR`	Only problems

Troubleshooting decision tree

```
Error?
├── "Error acquiring the state lock"
│      → someone/something else is applying, or a stale lock
│      → wait, or `terraform force-unlock <ID>` if truly stale
│
├── "AccessDenied" / "UnauthorizedOperation"
│      → IAM permissions issue → check the role's policy vs the action being called
│
├── "Error: Provider produced inconsistent result" / plan keeps changing
│      → likely a provider bug or a computed value depended on incorrectly → check provider changelog / GH iss
│
├── Unexpected destroy/replace in plan
│      → check for a ForceNew attribute change, or count/for_each re-indexing
│
├── "Resource already exists"
│      → resource exists in AWS but not in state → import it
│
└── Everything looks right but plan still shows changes
       → run with TF_LOG=DEBUG, check for attribute drift or defaults set server-side
```

PART 35 — AWS Authentication

How Terraform discovers AWS credentials (order of precedence)

1. Explicit `provider "aws" { access_key = ... secret_key = ... }` — **never do this in committed code**
2. Environment variables: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`
3. Shared credentials file: `~/.aws/credentials` (`aws configure`), optionally a named `profile`
4. `AssumeRole` via `provider "aws" { assume_role { role_arn = ... } }`
5. **Instance metadata** (EC2 instance profile / ECS task role / EKS IRSA / Lambda execution role) — the credentials attached to the compute Terraform itself runs on

```
provider "aws" {
  region  = "ap-south-1"
  profile = "my-sso-profile"     # local dev via aws configure sso
}
```

Production pattern — OIDC in CI/CD (no long-lived keys at all)

```
GitHub Actions / Jenkins / GitLab CI
        │  presents a short-lived OIDC token
        ▼
   AWS STS AssumeRoleWithWebIdentity
        │
        ▼
  Temporary credentials (15min–1hr), scoped by an IAM trust policy
```

```
# Example: GitHub Actions OIDC trust policy allows only a specific repo/branch to assume the role
```

WARNING — why hardcoding access keys is dangerous: long-lived static keys committed to Git or baked into CI variables are a top cause of cloud breaches (leaked keys get scraped by bots within minutes of a public commit). Always prefer OIDC/AssumeRole/instance-profile credentials that are short-lived and scoped.

BEST PRACTICE checklist: - Local dev: `aws configure sso` (temporary, personal, MFA-backed) - CI/CD: OIDC federation → AssumeRole, zero static keys stored anywhere - EC2/ECS/EKS running Terraform itself: instance profile / task role / IRSA - Never: static `AWS_ACCESS_KEY_ID` in a `.tf` file or committed `.tfvars`

PART 36 — Terraform + AWS (Core Services)

VPC & Networking

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block               = "10.0.1.0/24"
  availability_zone        = "ap-south-1a"
  map_public_ip_on_launch  = true
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
}

resource "aws_eip" "nat" { domain = "vpc" }

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public.id
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}
```

Security Groups & NACLs

```
resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id
  ingress { from_port = 443; to_port = 443; protocol = "tcp"; cidr_blocks = ["0.0.0.0/0"] }
  egress  { from_port = 0;   to_port = 0;   protocol = "-1"; cidr_blocks = ["0.0.0.0/0"] }
}
```

Security groups are **stateful** (return traffic auto-allowed); NACLs (`aws_network_acl`) are **stateless** (must allow both directions explicitly) and apply at the subnet level as a second layer of defense.

IAM

```
resource "aws_iam_role" "ec2_role" {
  name = "ec2-app-role"
```

```
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{ Action = "sts:AssumeRole", Effect = "Allow",
      Principal = { Service = "ec2.amazonaws.com" } }]
  })
}
resource "aws_iam_instance_profile" "ec2_profile" {
  role = aws_iam_role.ec2_role.name
}
```

EC2, Launch Template, ASG

```
resource "aws_launch_template" "web" {
  name_prefix   = "web-"
  image_id      = data.aws_ami.latest.id
  instance_type = "t3.micro"
  iam_instance_profile { name = aws_iam_instance_profile.ec2_profile.name }
  user_data = base64encode(templatefile("${path.module}/user_data.sh.tpl", { env = var.environment }))
}

resource "aws_autoscaling_group" "web" {
  desired_capacity   = 2
  min_size           = 2
  max_size           = 6
  vpc_zone_identifier = [aws_subnet.private_a.id, aws_subnet.private_b.id]
  target_group_arns   = [aws_lb_target_group.web.arn]
  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
}
```

ALB + Target Group

```
resource "aws_lb" "web" {
  load_balancer_type = "application"
  subnets             = [aws_subnet.public_a.id, aws_subnet.public_b.id]
  security_groups      = [aws_security_group.alb.id]
}
resource "aws_lb_target_group" "web" {
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id
  health_check { path = "/health" }
}
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.web.arn
  port              = 80
  default_action { type = "forward"; target_group_arn = aws_lb_target_group.web.arn }
}
```

RDS, S3, DynamoDB

```
resource "aws_db_instance" "main" {
  engine                 = "postgres"
  instance_class          = "db.t3.micro"
  allocated_storage        = 20
  db_subnet_group_name     = aws_db_subnet_group.main.name
  vpc_security_group_ids    = [aws_security_group.db.id]
  multi_az                = true
  skip_final_snapshot      = false
}

resource "aws_s3_bucket" "data" { bucket = "my-app-data-bucket" }
resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_dynamodb_table" "sessions" {
  name         = "sessions"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "session_id"
  attribute { name = "session_id"; type = "S" }
}
```

Route 53, CloudWatch, Lambda, ECR/ECS, KMS, Secrets Manager, SQS/SNS

```
resource "aws_route53_record" "app" {
  zone_id = var.zone_id
  name    = "app.example.com"
  type    = "A"
  alias { name = aws_lb.web.dns_name; zone_id = aws_lb.web.zone_id; evaluate_target_health = true }
}

resource "aws_cloudwatch_metric_alarm" "high_cpu" {
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 2
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EC2"
  period              = 300
  statistic           = "Average"
  threshold           = 80
}

resource "aws_lambda_function" "processor" {
  function_name = "process-events"
  role          = aws_iam_role.lambda.arn
  handler       = "index.handler"
  runtime       = "python3.12"
  filename      = "lambda.zip"
}
```

```
resource "aws_ecr_repository" "app" { name = "my-app" }

resource "aws_ecs_cluster" "main" { name = "app-cluster" }
resource "aws_ecs_service" "app" {
  cluster         = aws_ecs_cluster.main.id
  desired_count   = 2
  launch_type     = "FARGATE"
  task_definition = aws_ecs_task_definition.app.arn
}

resource "aws_kms_key" "main" { description = "app encryption key" }

resource "aws_secretsmanager_secret" "db_password" { name = "prod/db/password" }

resource "aws_sqs_queue" "jobs" { name = "job-queue" }
resource "aws_sns_topic" "alerts" { name = "alerts" }
```

PART 37 — Complete Production AWS Project (3-Tier Architecture)

```
                        Internet
                           │
                        Route 53
                           │
              Application Load Balancer (public subnets)
                           │
                Auto Scaling Group  (private subnets)
                     │        │
                    EC2      EC2   (application tier)
                           │
                RDS (Multi-AZ, private subnets, no public IP)

VPC (10.0.0.0/16)
├── Public Subnets  (10.0.1.0/24, 10.0.2.0/24) — 2 AZs
│   ├── ALB
│   └── NAT Gateway (per AZ, for outbound from private subnets)
│
└── Private Subnets (10.0.11.0/24, 10.0.12.0/24) — 2 AZs
    ├── EC2 (via ASG, behind ALB, IMDSv2 enforced)
    └── RDS (Multi-AZ, subnet group spans both private subnets)
```

Terraform dependency flow

```
module.vpc
   │  (vpc_id, subnet_ids)
   ▼
module.security_groups
   │  (sg_ids)
   ▼
module.alb  ──────────────┐
   │ (target_group_arn)   │
   ▼                      │
module.asg  ◄─────────────┘ (attaches to target group)
   │
   ▼
module.rds  (depends on private subnet group + db security group)
```

Root `main.tf` wires modules together:

```
module "vpc" {
  source = "./modules/vpc"
  # ...
}
module "security_groups" {
  source = "./modules/security-group"
  vpc_id = module.vpc.vpc_id
}
module "alb" {
  source            = "./modules/alb"
  public_subnet_ids = module.vpc.public_subnet_ids
```

```
  security_group_id = module.security_groups.alb_sg_id
}
module "asg" {
  source              = "./modules/ec2"
  private_subnet_ids  = module.vpc.private_subnet_ids
  target_group_arn    = module.alb.target_group_arn
  security_group_id   = module.security_groups.app_sg_id
}
module "rds" {
  source              = "./modules/rds"
  private_subnet_ids  = module.vpc.private_subnet_ids
  security_group_id   = module.security_groups.db_sg_id
}
```

This is exactly the kind of architecture worth building end-to-end as a portfolio project — it exercises networking, security, compute, load balancing, database, and module design all at once.

PART 38 — Terraform Project Structure

Beginner

```
terraform/
├── main.tf
├── variables.tf
├── outputs.tf
└── terraform.tfvars
```

Fine for learning / single-resource experiments.

Intermediate

```
terraform/
├── provider.tf
├── main.tf
├── variables.tf
├── outputs.tf
├── locals.tf
├── versions.tf
└── terraform.tfvars
```

Good for a single small real project with one environment.

Production

```
terraform/
├── environments/
│   ├── dev/       (backend.tf, main.tf calling modules, dev.tfvars)
│   ├── staging/
│   └── prod/
│
├── modules/
│   ├── vpc/
│   ├── ec2/
│   ├── alb/
│   └── rds/
│
└── README.md
```

Use this once you have >1 environment or >1 person touching the code — the separation prevents a `dev` change from ever accidentally reaching `prod`, and shared modules keep environments structurally consistent.

PART 39 — Terraform Security

Checklist

- **Secrets**: never hardcode; use `TF_VAR_*`, a secrets manager (AWS Secrets Manager/Vault) referenced via `data` source, or ephemeral resources (TF ≥1.10) so the value never lands in state.
- **`sensitive = true`**: hides values from CLI/plan output — **does not encrypt state**. Real protection = encrypted backend + tight IAM.
- **State security**: SSE-KMS on the S3 bucket, versioning enabled, bucket policy denies public access, IAM scoped to CI role + admins only.
- **IAM least privilege**: the role Terraform runs as should only have permissions for the resources it actually manages — not `AdministratorAccess`.
- **Backend encryption**: `encrypt = true` on the S3 backend block.
- **`.gitignore`**: exclude `*.tfstate*`, `.terraform/`, `*.tfvars` (except `.example` files), crash logs.
- **CI/CD secrets**: injected via the CI platform's secret store (GitHub Actions secrets, etc.), never printed in logs.
- **OIDC over static keys** (see Part 35).
- **Secret scanning**: run `gitleaks`/`trufflehog` in CI to catch accidentally committed secrets before merge.

WARNING: `sensitive = true` is a **UI/UX guard**, not encryption. Anyone with read access to the state file (S3 object, `terraform state pull`, etc.) can see the real value in plaintext JSON.

PART 40 — Terraform with Git

What to commit

- All `.tf` files
- `.terraform.lock.hcl` (pins provider versions for the whole team)
- `*.tfvars.example` (documents required variables without real values)
- `README.md`

What to NEVER commit

- `*.tfstate`, `*.tfstate.backup`
- `.terraform/` (downloaded plugin binaries — regenerated by `init`)
- Real `*.tfvars` containing secrets
- Any file with plaintext credentials

Professional `.gitignore`

```
# Terraform
**/.terraform/*
*.tfstate
*.tfstate.*
crash.log
crash.*.log
*.tfvars
*.tfvars.json
!*.tfvars.example
override.tf
override.tf.json
*_override.tf
*_override.tf.json
.terraformrc
terraform.rc
```

Workflow

- Feature branch → PR → CI runs `fmt -check`, `validate`, `plan` (posts plan as a PR comment) → human review of the *plan diff*, not just the code diff → merge → CD applies.

PART 41 — Terraform + CI/CD

```
Developer → Git → Pull Request
                        |
                        ▼
                        CI
              terraform fmt -check
              terraform validate
              terraform plan -out=tfplan   (posts diff as PR comment)
                        |
                  Code Review / Approval
                        |
                        ▼
              terraform apply tfplan
                        |
                        ▼
                       AWS
```

GitHub Actions example

```yaml
name: terraform
on: [pull_request, push]
permissions:
  id-token: write     # required for OIDC
  contents: read
jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gha-terraform
          aws-region: ap-south-1
      - run: terraform init
      - run: terraform fmt -check
      - run: terraform validate
      - run: terraform plan -out=tfplan
  apply:
    needs: plan
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: terraform apply -auto-approve tfplan
```

Jenkins / GitLab CI / CodePipeline

Same conceptual stages (`fmt`/`validate`/`plan`/approval/`apply`) — the tools differ only in syntax: - **Jenkins**: a `Jenkinsfile` with `sh 'terraform plan'` stages + an `input` step for manual approval. - **GitLab CI**: `.gitlab-ci.yml` with `plan` and `apply` jobs, using GitLab-managed Terraform state as a backend option. - **AWS CodePipeline**: CodeBuild stages running the same CLI commands,

often paired with CodeStar/OIDC for AWS auth.

Key CI/CD principles

- **Plan on every PR**, apply only after merge to the protected branch.
- **State locking** prevents two pipeline runs from racing.
- **Secrets** via OIDC (Part 35), never static keys in pipeline variables.
- **Environment promotion**: the same plan artifact (or same commit) flows dev → staging → prod, never re-planned differently per stage.

PART 42 — Terraform Testing

Layer	Tool	What it catches
Syntax	`terraform fmt` / `validate`	Formatting, basic HCL correctness
Linting	**TFLint**	Provider-specific best-practice violations, unused variables, deprecated syntax
Security scanning	**Checkov**, **Trivy** (also covers tfsec's former scope — tfsec merged into Trivy)	Misconfigurations: public S3 buckets, unencrypted volumes, overly permissive SGs
Policy-as-code	**OPA/Conftest**, or Sentinel (HCP Terraform)	Org-specific rules: "no resources outside approved regions," "all EC2 must have a cost-center tag"
Functional/unit testing	**`terraform test`** (native, `.tftest.hcl` files, Terraform ≥1.6)	Actually plans/applies against real or mocked providers and asserts on outputs

`terraform test` example

```
# tests/vpc.tftest.hcl
run "vpc_has_two_public_subnets" {
  command = plan
  assert {
    condition     = length(module.vpc.public_subnet_ids) == 2
    error_message = "Expected 2 public subnets"
  }
}
```

```
terraform test
```

PART 43 — Terraform Quality Tools

Tool	Purpose	When to use
`terraform fmt`	Formatting	Every change (pre-commit hook)
`terraform validate`	Syntax/config validation	Every change, every CI run
TFLint	Terraform-aware linting	CI, pre-commit
Checkov	IaC security/compliance scanning	CI
Trivy	Security + misconfiguration scanning (also container/image scanning)	CI
OPA/Conftest	Custom policy enforcement	Governance gate before apply
`terraform test`	Native functional testing	Module development, CI

PART 44 — Terraform Best Practices Checklist

Naming & Formatting - Consistent resource naming (`<project>-<env>-<resource>`), run `terraform fmt` via pre-commit hook.

Variables & Modules - Type every variable, describe every variable/output, validate where it matters. - Small, single-responsibility, versioned modules; prefer community modules for generic infra.

State & Backends - Remote backend always (S3/HCP Terraform); encryption + versioning + locking on. - Split state by blast radius (network / compute / data, or by environment).

Providers - Pin versions (`~>`); commit the lock file; upgrade deliberately with `-upgrade`.

Secrets & IAM - No static keys; OIDC/AssumeRole/instance profiles; least-privilege IAM for the Terraform execution role itself.

Git & CI/CD - `.gitignore` state/secrets; plan-on-PR, apply-after-merge; plan artifact = what gets applied.

Testing - `validate` + TFLint + Checkov/Trivy in CI at minimum; `terraform test` for critical modules.

Documentation - README per module (inputs/outputs/example usage); architecture diagram for the root config.

Environment Isolation - Separate AWS accounts for prod vs non-prod; directory-based environments over workspaces for real isolation.

Lifecycle & Drift - `prevent_destroy` on stateful/critical resources; restrict console write access to Terraform-managed resources; `-refresh-only` to consciously reconcile drift.

Disaster Recovery - Versioned state bucket (instant rollback); documented `force-unlock` and state-recovery runbook; regular `state pull` backups for extra safety.

PART 45 — Common Terraform Mistakes

#	Mistake	Why it happens	Correct approach
1	Hardcoding AWS credentials in `.tf`	Fastest way to "just get it working" locally	OIDC/profiles/instance roles (Part 35)
2	Committing `.tfstate` to Git	Not knowing it contains secrets/is regenerated	`.gitignore` it, use a remote backend
3	Not using remote state	Starting solo, never revisiting setup	Set up S3+locking from day one, even solo
4	Not pinning provider versions	Copy-pasted example configs omit constraints	Always set `version = "~> X.Y"`
5	Overusing `count` for dissimilar resources	Habit from other imperative tools	Use `for_each` with a map/set for stable identity
6	Incorrect `for_each` (passing a list instead of set/map)	`for_each` requires unique keys	`toset()`/convert to a map first
7	Using `-target` as routine workflow	"Just apply this one thing quickly"	Reserve for genuine emergencies only
8	Ignoring state locking errors (force-unlocking casually)	Impatience during a stuck CI run	Verify no other run is active before force-unlock
9	Bad module design (god-modules)	Trying to handle every possible case in one module	Small, composable, single-responsibility modules
10	Overusing modules for trivial resources	Cargo-culting "modules are best practice"	A single `aws_s3_bucket` doesn't need a module
11	Hardcoded values instead of variables	Quick prototyping never revisited	Parameterize anything that varies by environment
12	Missing/incorrect dependencies	Assuming implicit detection covers everything	Add `depends_on` for non-attribute dependencies
13	Ignoring drift indefinitely	No process for reviewing plan diffs regularly	Scheduled drift-detection plans in CI
14	Unsafe/accidental `destroy`	Running in the wrong directory/workspace	Separate state per env, `prevent_destroy`, approval gates
15	Exposing secrets via unprotected outputs	Forgetting `sensitive = true`	Mark sensitive outputs/variables; still protect state itself

#	Mistake	Why it happens	Correct approach
16	Wrong `lifecycle` rules (e.g., `prevent_destroy` blocking a legitimate rebuild)	Copy-pasted lifecycle blocks without understanding them	Understand each setting before applying it
17	Manually editing state with a text editor	Desperation during an incident	Use `state mv`/`rm`/import — never hand-edit JSON
18	One giant monolithic state for the whole org	Simplicity early on, never split later	Split state by blast radius before it becomes unmanageable
19	Not running `plan` before `apply` in production	Overconfidence / time pressure	Always read the plan, especially in prod
20	Deleting `.terraform.lock.hcl` to "fix" errors	Misunderstanding what the lock file does	`init -upgrade` deliberately instead
21	Using `terraform import` without matching HCL	Not realizing import doesn't generate config (legacy CLI import)	Use `-generate-config-out` or hand-write matching config first
22	No version control branching strategy for infra	Treating IaC less seriously than app code	Same PR/review discipline as application code
23	Secrets in `terraform.tfvars` committed by mistake	`.tfvars` looks like "just config"	`.gitignore` real `.tfvars`; commit only `.example`
24	Running `apply -auto-approve` locally in prod	Convenience	Reserve `-auto-approve` for CI with a prior reviewed plan
25	Not using workspaces *or* directories consistently — mixing approaches	Ad-hoc growth of the repo	Pick one environment strategy and standardize
26	Circular dependencies between modules	Two modules each needing the other's output	Refactor shared values into a third module or the root
27	Ignoring provider deprecation warnings	"It still works"	Address deprecations before forced upgrades break you
28	Not tagging resources consistently	Tags added ad-hoc per resource	Centralize via `default_tags` on the provider + `locals.common_tags`
29	Treating `sensitive = true` as real encryption	Misunderstanding the feature	Protect the backend itself; use secrets managers for true secrets

#	Mistake	Why it happens	Correct approach
30	Skipping `terraform fmt`/linting in CI	“It’s just formatting”	Enforce via pre-commit + CI gate — keeps diffs clean and reviewable

PART 46 — Troubleshooting Scenarios

1. `Error acquiring the state lock` Diagnosis: another `plan`/`apply` is running, or a previous run crashed and left a stale lock. Check who else might be running Terraform (CI dashboard, teammates). If genuinely stale: `terraform force-unlock <LOCK_ID>`.

2. AWS `AccessDenied` Diagnosis: the IAM identity Terraform is using lacks a permission. Read the exact action in the error (`iam:CreateRole`, etc.), check the role's attached policies, use IAM Policy Simulator or CloudTrail to confirm.

3. Terraform wants to destroy an existing EC2 instance unexpectedly Diagnosis: check plan output for `-/+` and which attribute forced replacement (often AMI ID, subnet, or a `ForceNew` field). Compare current `.tf` vs previous — did someone change `count`→`for_each` indices, or an AMI data source resolve to a new ID?

4. A resource exists in AWS but Terraform doesn't know about it ("already exists" on apply) Diagnosis: it was created manually or by another process. Fix: `import` block or `terraform import` to adopt it into state, then reconcile config.

5. Terraform state is corrupted (invalid JSON, missing fields) Diagnosis: check S3 bucket versioning for the last-good version and restore it, or `terraform state pull` a backup. As a last resort, rebuild state via careful `import` of each resource.

6. Provider version conflict Diagnosis: two modules require incompatible provider version ranges. Fix: align `required_providers` constraints across modules, or upgrade the older module.

7. `terraform plan` shows unexpected changes with no code change Diagnosis: likely drift (someone changed it in console) or a provider default changed after a provider upgrade. Run with `TF_LOG=DEBUG`, diff against `state show`.

8. `Error: Cycle` in dependency graph Diagnosis: two resources reference each other directly or indirectly. Break the cycle by extracting the shared value into a `local` or restructuring the reference.

9. `for_each` argument depends on resource attributes not known until apply Diagnosis: you can't use a computed value (like an ID generated during apply) as the `for_each` key — Terraform needs to know the *set of keys* during plan. Fix: use a value known before apply (e.g., a variable-driven map) instead.

10. Module version drift between environments Diagnosis: `dev` uses `module` source pinned to `v1.3.0`, `prod` still on `v1.1.0` — environments diverge silently. Fix: track module versions explicitly per environment and promote deliberately.

11. `terraform apply` timeout on RDS/EKS creation Diagnosis: some AWS resources take 10–40+ minutes; check provider `timeouts` block, don't assume it's hung.

12. Sensitive value leaked in CI logs Diagnosis: an output or error message printed a value that should've been `sensitive`. Fix: mark it sensitive, scrub CI logs, rotate the leaked credential immediately.

13. `Error: Invalid count argument` — count depends on a resource attribute Diagnosis: same root cause as #9 but for `count`. The count *number* must be knowable at plan time.

14. Backend re-init fails after changing bucket/key Diagnosis: forgot `-migrate-state` or `-reconfigure`. Use `-migrate-state` when you want old state carried over, `-reconfigure` to start fresh.

15. Two engineers' local applies conflict / "who changed what" Diagnosis: no remote backend / no locking. Root cause fix: adopt S3 + DynamoDB (or S3-native) locking immediately.

16. IAM role just created isn't recognized by a dependent resource (eventual consistency) Diagnosis: AWS IAM propagation lag. Fix: add `depends_on`, or in stubborn cases a short `time_sleep` resource from the `time` provider.

17. `terraform destroy` fails because of `prevent_destroy` Diagnosis: working as intended — remove the `lifecycle` block deliberately, re-plan, confirm, then destroy.

18. Plan shows changes to a field you never set Diagnosis: provider/AWS applies a default value server-side; either explicitly set it in your config to match, or `ignore_changes` it if it's genuinely managed elsewhere (e.g., by autoscaling).

19. `Error: Unsupported argument` after a provider upgrade Diagnosis: the provider deprecated/renamed an argument in a new major version — check the provider's upgrade guide/changelog before bumping `required_providers`.

20. CI pipeline applies stale plan after a long approval wait Diagnosis: infra drifted between `plan` and `apply` (long manual approval gate). Fix: keep the window short, or re-plan if the approval takes too long; some teams add a freshness check.

PART 47 — Advanced Terraform

- **Provider aliases + multi-account**: see Part 48.
- **Preconditions/postconditions**: assert invariants (Part 21) — fail loud instead of deploying silently-wrong infra.
- **Import/moved blocks**: declarative, reviewable refactors (Parts 22–23).
- **Checks**: post-apply health assertions independent of any one resource (Part 4).
- **Terraform internals**: Core builds a DAG → walks it with bounded parallelism → each node's apply is a provider RPC call → state is a *cache* of last-known real-world truth, not the source of truth itself (the cloud is).
- **Refresh behavior**: modern Terraform refreshes as part of `plan`/`apply` by default; `-refresh=false` skips it (faster, but risks stale-drift blindness — use only when you're sure nothing changed out-of-band).
- **Parallelism**: `terraform apply -parallelism=20` raises the default (10) concurrent operations — useful for large graphs, but can hit cloud API rate limits.

PART 48 — Multi-Account AWS

```
AWS Organization
│
├── Management Account   (Organizations, billing, SCP policies)
├── Security Account     (CloudTrail, GuardDuty aggregation, audit roles)
├── Dev Account
├── Staging Account
└── Production Account
```

```
provider "aws" {
  alias  = "dev"
  region = "ap-south-1"
  assume_role { role_arn = "arn:aws:iam::111111111111:role/terraform-dev" }
}
provider "aws" {
  alias  = "prod"
  region = "ap-south-1"
  assume_role { role_arn = "arn:aws:iam::222222222222:role/terraform-prod" }
}

module "dev_vpc" {
  source    = "./modules/vpc"
  providers = { aws = aws.dev }
}
module "prod_vpc" {
  source    = "./modules/vpc"
  providers = { aws = aws.prod }
}
```

- **Account isolation =** the strongest blast-radius boundary (separate IAM, service limits, billing).
- Each account typically has **its own state file** (often in a shared, centrally-owned state bucket in the Security/Management account with cross-account bucket policies).
- CI/CD pipeline assumes a per-environment role via OIDC, scoped so the `dev` pipeline literally cannot assume the `prod` role.

PART 49 — Terraform Performance

- **Parallelism**: default 10 concurrent ops; tune with `-parallelism=n` for very large graphs (watch cloud API rate limits).
- **Large state files**: slow `plan`/`apply`, higher blast radius, harder to reason about — split by service/environment (Part 17's advice).
- **Module design**: deeply nested modules-of-modules add planning overhead; keep the tree reasonably flat.
- **Excessive data sources**: every `data` block is a real API call on every `plan` — avoid re-querying the same lookup in many places (compute once in a shared module/locals and pass it down).
- **Unnecessary `depends_on`**: forces artificial serialization, killing parallelism — only add it when truly needed.
- **`-refresh=false`** for rapid iterative dev-loop plans when you're confident nothing drifted (never for the final production plan).

PART 50 — Terraform Production Architecture (Enterprise)

```
Terraform (org-wide repo layout)
│
├── networking/     (VPC, subnets, transit gateway — owned by platform team)
├── security/        (SGs, NACLs, KMS keys, WAF — owned by security team)
├── iam/              (roles, policies, SSO — owned by platform team)
├── compute/          (ASGs, EKS clusters — owned by platform team)
├── load-balancing/   (ALBs, target groups — owned by platform team)
├── database/          (RDS, DynamoDB — owned by data team)
├── monitoring/         (CloudWatch, alerting — owned by SRE team)
└── application-infra/  (per-service ECS/EKS resources — owned by app teams)
```

Each layer has **its own state**, is applied in dependency order (networking → security/IAM → compute → app), and is often owned by a different team with scoped IAM permissions matching their layer — this is how large orgs keep Terraform safe at scale: small blast radii, clear ownership, explicit cross-layer references via remote state data sources (`terraform_remote_state`) or SSM Parameter Store outputs.

PART 51 — Terraform + DevOps Ecosystem

Tool	Terraform's relationship to it
Linux/Bash	Terraform CLI runs on it; user_data/provisioner scripts are usually bash
Python	Used for custom automation around Terraform (wrapper scripts, CI glue), not required by TF itself
Git/GitHub	Source of truth for `.tf` code, PR-driven review workflow
Packer	Builds the **golden AMI**; Terraform then references that AMI ID
Docker	Builds container images; Terraform provisions the ECS/EKS/Fargate infra that *runs* them
Jenkins/GitHub Actions	Orchestrates the `init/plan/apply` pipeline
Kubernetes	Terraform can create the **cluster** (EKS) and cluster-level resources (via the `kubernetes`/`helm` providers), but app deployments inside the cluster are usually Helm/kubectl/GitOps (Argo CD), not Terraform
Ansible	Occasionally configures *inside* a VM after Terraform creates it — increasingly replaced by immutable AMIs (Packer) or containers
Prometheus/Grafana	Terraform can provision the *infrastructure* (EC2/EKS/managed AMP/AMG) hosting them; dashboards/alerts config is usually managed by their own tools or the `grafana` Terraform provider

Terraform + Packer flow

```
Packer  →  builds a Golden AMI (app + OS baked in, immutable)
   ↓
Terraform  →  references that AMI ID in a Launch Template
   ↓
Auto Scaling Group  →  launches instances from it
```

Rule of thumb: Terraform = "what infrastructure exists." Packer/Docker = "what runs on/in it." Ansible = "how to mutate a running thing" (increasingly avoided in immutable-infra shops). Kubernetes/Helm/Argo CD = "what application workloads run inside the cluster Terraform built."

PART 52 — Terraform vs Other Tools (Detailed)

Comparison	Key difference
Terraform vs Ansible	Terraform = declarative provisioning + state tracking; Ansible = imperative, agentless config management, no built-in state file. Often used together: TF provisions, Ansible configures (or is replaced by Packer for immutability).
Terraform vs CloudFormation	CFN is AWS-native and free but AWS-only; Terraform is multi-cloud with a larger community module ecosystem and its own state model (CFN manages state internally via stacks).
Terraform vs Pulumi	Same declarative-provisioning goal; Pulumi uses real programming languages (loops/conditionals/tests are native, not HCL functions) at the cost of a steeper "which SDK version" surface and different team skill requirements.
Terraform vs AWS CDK	CDK is imperative code that *synthesizes* CloudFormation templates — AWS-only, inherits CFN's state model under the hood.
Terraform vs Packer	Different jobs entirely — Packer builds images, Terraform provisions infra; they're complementary, not competitors.
Terraform vs Kubernetes manifests/Helm	Terraform is best for the *cluster and cloud resources around it*; K8s-native tools (kubectl/Helm/Argo CD) are usually better for the *workloads running inside* the cluster, since they're built for that continuous-reconciliation use case.

INTERVIEW TIP: Frame these as "complementary, not competitive" — interviewers like hearing that you understand tool boundaries, not just "Terraform is better."

PART 53 — Interview Preparation

Beginner

Q1. What is Terraform? An open-source, declarative Infrastructure-as-Code tool that provisions and manages infrastructure across many providers using HCL.

Q2. What is HCL? HashiCorp Configuration Language — a declarative, block-based language designed to be both human-readable and machine-parseable.

Q3. What is the Terraform workflow? `init` → `plan` → `apply` (→ `destroy` when needed) — see Part 1's diagram.

Q4. What is a provider? A plugin translating HCL resource blocks into API calls for a specific platform (e.g., `hashicorp/aws`).

Q5. Difference between `resource` and `data` blocks? `resource` creates/manages infra lifecycle; `data` only reads existing infra.

Q6. What is Terraform state, and why is it needed? A JSON file mapping config to real resource IDs so Terraform can compute diffs without re-discovering everything from scratch every run (Part 15).

Q7. What does `terraform plan` do? Refreshes state, diffs config vs. state vs. real infra, and prints the actions Terraform would take without making changes.

Q8. What is a variable, and how do you set one? An input to a module; set via default, `.tfvars`, `-var`, `TF_VAR_*`, or CLI flags — precedence in Part 6.

Q9. What are locals? Named expressions computed inside a module for reuse/DRY-ing config — not settable from outside (Part 7).

Q10. What is an output? A value exposed after apply, readable via `terraform output` or by a parent module (Part 8).

Q11–20 (rapid-fire, know the one-line answer): What is `.terraform.lock.hcl`? (pins provider versions/checksums) · What is `terraform fmt` for? (canonical formatting) · What is `terraform validate`? (syntax/consistency check, no API calls) · What's the default backend? (local) · What is a module? (any directory of `.tf` files, reusable via a `module` block) · What is `count`? (numeric-indexed resource repetition) · What is `for_each`? (key-based resource repetition, stable identity) · What does `depends_on` do? (explicit dependency Terraform can't infer) · What is `terraform destroy`? (removes all managed resources) · What's a `.tfvars` file? (variable value assignments).

Intermediate

Q21. Difference between `count` and `for_each`? See Part 11's table — `count` uses numeric indices (unsafe reordering); `for_each` uses stable string keys from a map/set.

Q22. What is state locking, and why does it matter? Prevents two concurrent `apply`s from corrupting the same state file; implemented via DynamoDB (or S3-native locking) on the S3 backend.

Q23. What happens if you delete the state file? Terraform loses all knowledge of what it manages — next `apply` would try to recreate everything from scratch, likely erroring on "already exists" resources. Recovery = restore from a versioned backend or re-`import` everything.

Q24. How do you handle drift? Detect via `plan` (shows the diff automatically); reconcile by either updating `.tf` to match reality or `apply`-ing to revert it; use `-refresh-only` to consciously

accept drift into state.

Q25. How do you import existing resources? Modern: an `import` block + `plan -generate-config-out`. Legacy: `terraform import <addr> <id>` with a hand-written matching resource block (Part 22).

Q26. How do you manage secrets in Terraform? Never hardcode; use env vars/`TF_VAR_*`, secrets managers referenced via data sources, ephemeral resources (TF ≥1.10), and protect the state backend itself (secrets end up there regardless of `sensitive`).

Q27. How do you manage multiple environments? Directory-based environments + separate state (preferred for real isolation) and/or separate AWS accounts; workspaces are a lighter-weight alternative for smaller variations (Parts 27–28).

Q28. Why use remote state? Team collaboration, locking, encryption at rest, versioned history/rollback, avoids "it works on my machine" local-state drift.

Q29. Why use modules? Reusability, consistency across environments, encapsulated complexity behind a clean interface, easier testing/versioning.

Q30. How does Terraform determine dependency order? Builds a DAG from implicit (attribute references) and explicit (`depends_on`) dependencies; independent branches run in parallel.

Q31–50 (know these well): What is `terraform_remote_state`? · Difference between implicit and explicit dependencies? · What is a provider alias, and when do you need one? (multi-region/multi-account) · What is `lifecycle { create_before_destroy }` for? · What does `prevent_destroy` protect against? · What's the difference between `list` and `set`? · Why does `for_each` require a set/map, not a list? · What is a `dynamic` block? · What is `terraform console` used for? · What's the difference between `terraform refresh` and `plan`? (refresh folded into plan/apply by default now) · What's a `moved` block for? · Difference between `state mv` and a `moved` block? (both remap addresses; `moved` is committed/reviewable, `state mv` is a manual one-off) · What is `ignore_changes` for? · What's a `check` block? · What is `templatefile()` used for? · What's the difference between a provider and a module? · What is the Terraform Registry? · What's a private module registry for? · What does `-target` do, and why avoid it routinely? · What's the difference between Terraform CLI and HCP Terraform?

Advanced

Q51. What happens if `apply` fails halfway through? Already-applied resources remain tracked in state; the failed/remaining resources are retried on the next `apply` — Terraform doesn't roll back successful steps automatically (Part 25).

Q52. How do you safely rename a resource without destroying it? `moved` block (preferred, reviewable) or `terraform state mv` (Part 23) — never just rename in code and reapply, which would destroy-then-recreate.

Q53. What is `create_before_destroy`, and when do you need it? Builds the replacement before destroying the old one — needed for zero-downtime swaps of uniquely-named or in-use resources (e.g., launch templates behind an ASG).

Q54. What is a provider alias, and how does multi-region/multi-account Terraform work? Multiple `provider` blocks with `alias`, referenced per-resource/module via the `provider =` meta-argument or `providers = {}` map on module calls (Parts 11, 48).

Q55. How does Terraform authenticate with AWS in CI/CD, and why avoid static keys? OIDC federation → `AssumeRoleWithWebIdentity` → short-lived STS credentials; static keys are long-lived and a major leak/breach risk (Part 35).

Q56. How would you design Terraform for a production organization? Layered state by blast radius/team ownership, per-environment AWS accounts, shared versioned modules, CI/CD with plan-on-PR/apply-after-merge, policy-as-code gates, encrypted+versioned remote state (Part 50).

Q57–80 (advanced topics to be ready to discuss in depth): Explain the Terraform plugin/RPC architecture · How does the DAG enable parallelism, and what limits it? · What's the difference between `precondition` and `postcondition`? · How do ephemeral resources avoid writing secrets to state (TF ≥1.10)? · How would you split a monolithic state file safely? (`state mv` / `moved` between configs, careful sequencing) · What's the risk of `-target`, concretely? · How do you handle a provider major-version upgrade safely? · What's the role of `.terraform.lock.hcl` in supply-chain security (checksums)? · How do you test a Terraform module? · What's Sentinel/OPA, and where does it run in the workflow? · How do you do zero-downtime infrastructure changes with Terraform? · How would you debug "plan keeps showing the same diff every run"? · What's the tradeoff of splitting state very finely vs. keeping it coarse? · Explain `terraform_remote_state` as a cross-state data source and its downsides (tight coupling) vs. SSM Parameter Store as an alternative.

Scenario-Based (sample — practice explaining your reasoning out loud)

- "Terraform wants to destroy and recreate your production RDS instance — what do you check, and what do you do?" → check plan's replacement reason (ForceNew attribute), verify `prevent_destroy`/`skip_final_snapshot`, take a manual snapshot before proceeding regardless, confirm with the team.
- "Two engineers ran `apply` at the same time and now state looks wrong — walk me through recovery." → check state lock history, compare state file versions in the versioned S3 bucket, restore the last known-good version, re-run `plan` to confirm reality matches.
- "A teammate accidentally deleted a security group in the console — how does Terraform respond, and what do you do?" → next `plan` shows it as a create (drift); decide whether to `apply` to recreate it or accept the change into code if it was intentional.
- "You need to move 50 resources into a new module structure without downtime — what's your approach?" → `moved` blocks generated systematically (often via a script mapping old→new addresses), tested against a `plan` showing **zero** create/destroy actions, only address changes.
- "How do you roll out a Terraform change across 20 microservice teams without breaking anyone?" → shared versioned modules with semantic versioning, changelogs, a deprecation window, and per-team upgrade PRs rather than a forced simultaneous change.

Production/SRE

- "How do you prevent a bad `apply` from taking down production?" → plan review gate, policy-as-code checks, `prevent_destroy` on critical resources, canary/staged rollout via environment promotion, monitoring + alerting tied to the change.
- "How do you audit who changed what infrastructure and when?" → Git history (PRs) + CI/CD apply logs + CloudTrail (for any out-of-band changes) + versioned state bucket history.
- "What's your incident response if Terraform state is lost entirely?" → restore from versioned backend backup; if truly gone, systematically `import` every resource, starting with the most critical, verifying each with `plan` showing no diff.
- "How do you handle a resource that Terraform can't cleanly manage (some manual, some automated fields)?" → `ignore_changes` on the specific attributes managed elsewhere (e.g., ASG `desired_capacity` managed by a scaling policy).

PART 54 — Hands-On Labs (Progressive Curriculum)

Lab	Objective	Key commands/concepts
1. Install Terraform	Get the CLI working, verify version	`terraform version`, tfenv for version management
2. First AWS resource	Provision one `aws_instance`, understand init/plan/apply	`init`, `plan`, `apply`, `destroy`
3. Variables	Parameterize instance type/region via `.tfvars`	`variable`, `-var-file`
4. Outputs	Expose instance public IP	`output`, `terraform output`
5. Data sources	Look up the latest AMI dynamically instead of hardcoding	`data "aws_ami"`
6. Loops	Create 3 instances with `for_each` over a set of names	`for_each`, `each.key`
7. Conditions	Toggle instance size based on an `environment` variable	ternary `? :`
8. VPC	Build a VPC with public/private subnets, IGW, route tables	`aws_vpc`, `aws_subnet`, `aws_route_table`
9. EC2 in the VPC	Launch an instance into a private subnet	subnet/SG wiring
10. Security Groups	Restrict SSH to your IP, HTTP from ALB only	`aws_security_group`, ingress/egress
11. ALB	Front the instance with an Application Load Balancer	`aws_lb`, `aws_lb_target_group`, `aws_lb_listener`
12. Auto Scaling	Replace standalone EC2 with a Launch Template + ASG	`aws_launch_template`, `aws_autoscaling_group`
13. RDS	Add a Multi-AZ Postgres instance in private subnets	`aws_db_instance`, `aws_db_subnet_group`
14. Modules	Refactor Labs 8–13 into `modules/vpc`, `modules/ec2`, `modules/alb`, `modules/rds`	module composition (Part 19)
15. Remote backend	Move state to S3 + locking	`backend "s3"`, `init -migrate-state`
16. State management	Practice `state list/show/mv/rm`, `moved` blocks	Part 16, 23
17. Import	Manually create a bucket in the console, then import it	`import` block + `-generate-config-out`
18. Multi-environment	Split into `environments/dev` and `environments/prod` sharing modules	Part 28 structure
19. CI/CD	Wire up GitHub Actions: fmt/validate/plan on PR, apply on merge	Part 41
20. Production-grade capstone	Full 3-tier architecture (Part 37) with modules, remote state, CI/CD, and security scanning in the pipeline	Everything combined

(Each lab follows the same pattern: write config → `plan` → `apply` → verify in AWS console → `destroy` before starting the next lab to avoid cost. Always double-check `terraform plan` output

before `apply`*, and run* `terraform destroy` *at the end of every lab session.)*

PART 55 — Real-World Projects (Resume-Worthy)

Project 1 — Simple EC2 Deployment Single EC2 instance with a security group and an Elastic IP, parameterized by variables. *Resume line:* "Provisioned a parameterized AWS EC2 environment using Terraform with variable-driven configuration and remote state."

Project 2 — VPC + EC2 Architecture Custom VPC, public/private subnets across 2 AZs, IGW/NAT, EC2 in private subnet reachable via a bastion or SSM Session Manager. *Resume line:* "Designed a multi-AZ AWS network architecture in Terraform, including custom VPC, subnetting, and secure instance access via SSM."

Project 3 — Highly Available Web Application ALB + Auto Scaling Group + Launch Template + target-tracking scaling policy, health checks, across 2 AZs. *Resume line:* "Built a self-healing, auto-scaling web tier behind an Application Load Balancer using Terraform-managed Launch Templates and ASGs."

Project 4 — Production-Style 3-Tier Architecture Full stack from Part 37: VPC, ALB, ASG, Multi-AZ RDS, modularized, remote state, IAM least-privilege roles. *Resume line:* "Architected and codified a production-grade 3-tier AWS application (web/app/DB tiers) as reusable Terraform modules with remote state and IAM least-privilege access."

Project 5 — Complete DevOps Pipeline Packer-built golden AMI → Terraform ASG/ALB/RDS → S3 for artifacts → CloudWatch alarms/dashboards → IAM roles → GitHub Actions CI/CD with OIDC, `fmt`/`validate`/`plan`-on-PR/`apply`-on-merge, and Checkov security scanning. *Resume line:* "Implemented an end-to-end infrastructure delivery pipeline combining Packer, Terraform, and GitHub Actions with OIDC authentication, automated security scanning, and environment promotion from dev to production."

This progression (and Project 5 in particular) maps directly onto skills already listed as hands-on experience: **Docker, Jenkins, ECR, ECS Fargate, and GitHub Actions** — a natural next step is swapping GitHub Actions for the existing Jenkins/ECS Fargate experience to build a portfolio piece that mirrors real internship work.

PART 56 — Cheat Sheets

1. CLI: `init` `validate` `fmt` `plan` `apply` `destroy` `show` `output` `providers` `version` `graph` `console` `get` `force-unlock` `state ...` `import` `workspace ...` `test`

2. HCL syntax: `block "label" { arg = value }`, comments `# // /* */`, heredoc `<<-EOF ... EOF`, interpolation `"${expr}"`

3. Variables: `variable "x" { type, default, description, sensitive, validation }`; precedence: CLI `-var`/`-var-file` > `*.auto.tfvars` > `terraform.tfvars` > `TF_VAR_*` > `default`

4. Data types: `string` `number` `bool` `list()` `set()` `map()` `object({})` `tuple([])` `null`

5. Functions: `merge` `concat` `flatten` `join` `split` `lookup` `contains` `length` `keys` `values` `toset` `tolist` `tomap` `try` `can` `coalesce` `distinct` `sort` `element` `cidrsubnet` `jsonencode` `jsondecode` `templatefile` `file`

6. Expressions: ternary `cond ? a : b`, `for` expr `[for x in y : x]` / `{for x in y : k => v}`, splat `x[*].attr`

7. Meta-arguments: `count` `for_each` `depends_on` `provider` `lifecycle`

8. Lifecycle: `create_before_destroy` `prevent_destroy` `ignore_changes` `replace_triggered_by` `precondition` `postcondition`

9. State: `terraform.tfstate` (local truth), `state list/show/mv/rm/pull/push`, locking via DynamoDB or S3-native, never hand-edit

10. Modules: `module "name" { source = "..." version = "~>" ... }`; root vs child; local/git/registry sources

11. AWS provider essentials: `provider "aws" { region }`, `required_providers { aws = { source = "hashicorp/aws" } }`, auth via profile/OIDC/instance role

12. Debugging: `TF_LOG=DEBUG`, `TF_LOG_PATH`, `terraform console`, `terraform graph`

13. Git: commit `.tf`, `.terraform.lock.hcl`, `*.tfvars.example`; ignore `.tfstate*` `.terraform/` real `*.tfvars`

14. CI/CD: `fmt -check` → `validate` → `plan -out` → review → `apply <planfile>`; OIDC for auth; lock state during runs

15. Security: no hardcoded secrets, `sensitive = true` (UI only, not encryption), encrypted+versioned state backend, least-privilege IAM

16. Troubleshooting: state lock stuck → `force-unlock`; AccessDenied → check IAM action; unexpected replace → check ForceNew attribute; resource exists → `import`; corrupted state → restore from versioned backend

PART 57 — Command Quick Reference

Command	Purpose	Example
`terraform init`	Initialize backend + providers	`terraform init -upgrade`
`terraform fmt`	Format code	`terraform fmt -recursive`
`terraform validate`	Validate syntax	`terraform validate`
`terraform plan`	Preview changes	`terraform plan -out=tfplan`
`terraform apply`	Apply changes	`terraform apply tfplan`
`terraform destroy`	Tear down	`terraform destroy -target=aws_instance.x`
`terraform state`	Inspect/edit state	`terraform state list`
`terraform import`	Adopt existing infra	`terraform import aws_s3_bucket.b my-bucket`
`terraform workspace`	Manage workspaces	`terraform workspace new dev`
`terraform console`	Expression REPL	`terraform console`

PART 58 — Revision Notes

1-Day Revision (bare minimum)

- Workflow: `init → plan → apply → destroy`
- State exists to map config → real resource IDs; always remote + locked in production
- `count` (numeric, unstable index) vs `for_each` (keyed, stable) — know this cold
- `variable`/`local`/`output` roles
- `resource` vs `data`
- Remote backend = S3 + locking (DynamoDB or S3-native)
- Never commit state or secrets; `.gitignore` basics
- Modules = reusable folders with their own variables/outputs

3-Day Revision

Add to Day 1: - Meta-arguments in full (`depends_on`, `lifecycle` settings) - Import workflow (block + CLI) - `moved` blocks / `state mv` for safe refactors - Drift: what it is, how `plan` surfaces it, `-refresh-only` - AWS auth chain (profile → OIDC → instance role) and why static keys are bad - Core AWS resources: VPC, EC2/ASG, ALB, RDS, S3, IAM — be able to sketch the 3-tier diagram from memory - Environment strategy: directories + accounts > workspaces alone

7-Day Revision (full schedule)

- **Day 1–2**: Fundamentals, HCL, blocks, providers, variables/locals/outputs, resources/data sources
- **Day 3**: Meta-arguments, expressions/functions, dynamic blocks, dependency graph
- **Day 4**: State deep-dive, state commands, remote backend, drift
- **Day 5**: Modules, lifecycle, import, moving/refactoring
- **Day 6**: AWS practical (VPC → RDS build-out), project structure, security, Git, CI/CD
- **Day 7**: Testing/quality tools, best practices, common mistakes, troubleshooting scenarios, interview questions, cheat sheets

Interview-Day Revision (most likely to come up)

1. Terraform workflow + what each command does internally
2. What state is, why it exists, and how locking/remote backends work
3. `count` vs `for_each`, with the “why” (stable identity)
4. How you’d import an existing resource and reconcile config
5. How you’d structure a multi-environment, multi-account setup
6. How AWS auth works in CI/CD (OIDC) and why static keys are avoided
7. What modules are for and how you’d design one
8. How a bad `apply` is prevented in production (plan review, policy-as-code, `prevent_destroy`)
9. One clear troubleshooting story (state lock, drift, or a forced replacement) told start to finish
10. Terraform vs Ansible/CloudFormation/Pulumi — one sentence each, “complementary not competing”

PART 59 — Learning Roadmap

```
Level 1 — Terraform Basics
  Learn: init/plan/apply/destroy, HCL syntax basics
  Practice: Labs 1–2
  Build: a single EC2 instance
  Ready for Level 2 when: you can explain the workflow diagram from memory

Level 2 — HCL + Resources
  Learn: blocks, resource/data, dependencies
  Practice: Labs 3–5
  Build: EC2 with a dynamic AMI lookup
  Ready when: you can predict plan output before running it

Level 3 — Variables + Functions
  Learn: variable types, precedence, locals, core functions, expressions
  Practice: Labs 6–7
  Build: a parameterized, environment-aware config
  Ready when: you're comfortable writing a `for` expression unaided

Level 4 — State + Modules
  Learn: state internals, state commands, module design
  Practice: Labs 14–17
  Build: refactor an earlier project into modules with imported existing infra
  Ready when: you can explain state locking and drift to someone else

Level 5 — AWS Infrastructure
  Learn: VPC/EC2/ALB/ASG/RDS/IAM in depth
  Practice: Labs 8–13
  Build: Project 3 (HA web app)
  Ready when: you can sketch the 3-tier architecture from memory

Level 6 — Production Terraform
  Learn: remote backends, environment strategy, multi-account
  Practice: Lab 18
  Build: Project 4 (3-tier, modularized, multi-env)
  Ready when: dev/staging/prod are cleanly separated with shared modules

Level 7 — CI/CD + Security
  Learn: OIDC auth, plan-on-PR/apply-on-merge, secrets handling, TFLint/Checkov
  Practice: Lab 19
  Build: Project 5 (full pipeline)
  Ready when: a PR you open triggers an automatic, reviewable plan

Level 8 — Advanced Terraform
  Learn: preconditions/postconditions, ephemeral resources, performance tuning, testing (`terraform test`)
  Practice: write `.tftest.hcl` for an existing module
  Ready when: you can debug an unfamiliar plan diff systematically, not by guessing

Level 9 — Enterprise Architecture
  Learn: layered state by team ownership, policy-as-code (OPA/Sentinel), org-wide module governance
  Build: Lab 20 capstone — the full production-grade AWS infrastructure end to end
  Ready when: you could explain, to a hiring manager, how you'd structure Terraform for a 50-engineer org
```

Given hands-on experience already with Docker, Jenkins, ECR, ECS Fargate, and GitHub Actions, Levels 5–7 are the highest-leverage next step — they connect directly into that existing toolchain (Terraform provisioning the ECS Fargate infra, Jenkins/GitHub Actions driving the pipeline) rather than starting from an unrelated stack.